Has you seen my files?

Attack on a server utilising the haserl framework.

Difficulty: Hard

Ben Poulton

Abertay HackSoc Term 1 CTF 2025/26

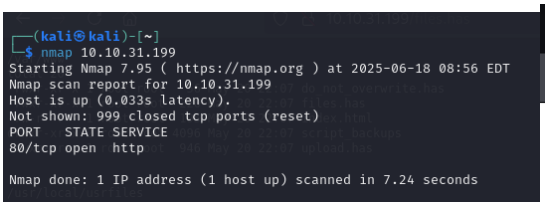
.

1 INTRODUCTION

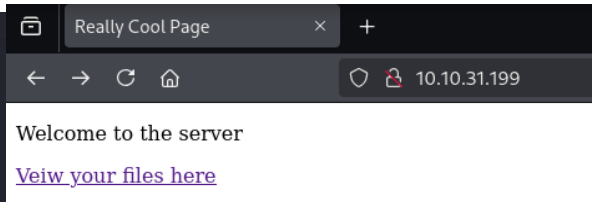
“Has you seen my files?” is a challenge centered around a web server utilising the Haserl CGI framework. This web server contains a page that has file listings, and a page that allows for the users to upload their own files, specifically to the /usr/local/usrfiles folder. By utilising file path traversal within the upload functionality, the user can upload their own haserl scripts to be run on the server, and therefore get access to flag.txt

2 SOLUTION

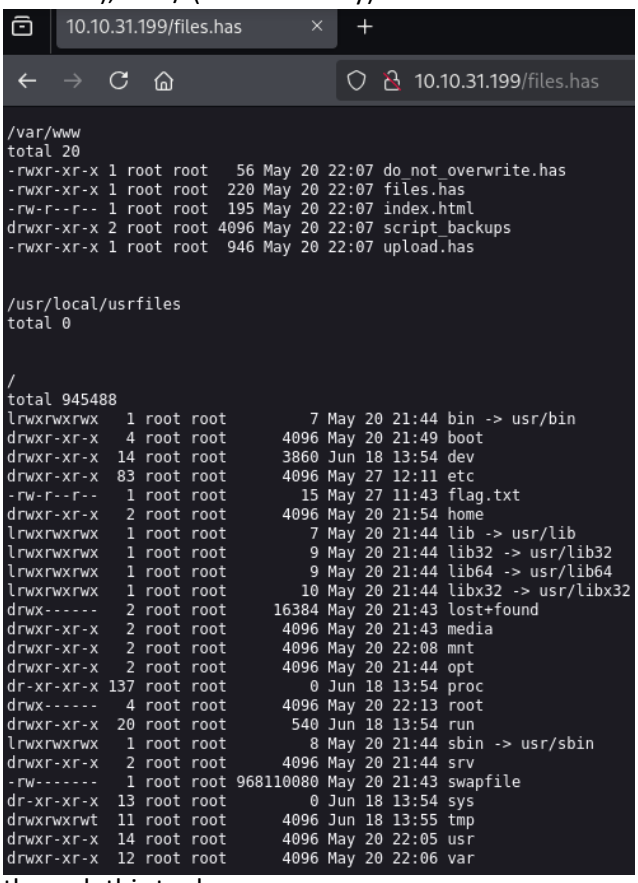
An initial NMAP scan of the server shows a http service open on port 80. Navigating to the server's IP address, they are shown a simple webpage with a hyperlink that states "View your files here"



```
┌──(kali㉿kali)-[~]
└─$ nmap 10.10.31.199
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-18 08:56 EDT
Nmap scan report for 10.10.31.199
Host is up (0.033s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 7.24 seconds
```



This link leads to the page files.has, which provides the user with the outputs of an ls -l command on the directories /var/www (web file location), /usr/local/usrfiles (user file location), and / (root directory)




```
/var/www
total 20
-rwxr-xr-x 1 root root   56 May 20 22:07 do_not_overwrite.has
-rwxr-xr-x 1 root root  220 May 20 22:07 files.has
-rw-r--r-- 1 root root  195 May 20 22:07 index.html
drwxr-xr-x 2 root root 4096 May 20 22:07 script_backups
-rwxr-xr-x 1 root root  946 May 20 22:07 upload.has


/usr/local/usrfiles
total 0


/
total 945488
lrwxrwxrwx   1 root root         7 May 20 21:44 bin -> usr/bin
drwxr-xr-x   4 root root      4096 May 20 21:49 boot
drwxr-xr-x  14 root root      3860 Jun 18 13:54 dev
drwxr-xr-x  83 root root      4096 May 27 12:11 etc
-rw-r--r--   1 root root        15 May 27 11:43 flag.txt
drwxr-xr-x   2 root root      4096 May 20 21:54 home
lrwxrwxrwx   1 root root         7 May 20 21:44 lib -> usr/lib
lrwxrwxrwx   1 root root         9 May 20 21:44 lib32 -> usr/lib32
lrwxrwxrwx   1 root root         9 May 20 21:44 lib64 -> usr/lib64
lrwxrwxrwx   1 root root        10 May 20 21:44 libx32 -> usr/libx32
drwx------   2 root root     16384 May 20 21:43 lost+found
drwxr-xr-x   2 root root      4096 May 20 21:43 media
drwxr-xr-x   2 root root      4096 May 20 22:08 mnt
drwxr-xr-x   2 root root      4096 May 20 21:44 opt
dr-xr-xr-x 137 root root         0 Jun 18 13:54 proc
drwx------   4 root root      4096 May 20 22:13 root
drwxr-xr-x  20 root root       540 Jun 18 13:54 run
lrwxrwxrwx   1 root root         8 May 20 21:44 sbin -> usr/sbin
drwxr-xr-x   2 root root      4096 May 20 21:44 srv
-rw-------   1 root root 968110080 May 20 21:43 swapfile
dr-xr-xr-x  13 root root         0 Jun 18 13:54 sys
drwxrwxrwt  11 root root      4096 Jun 18 13:55 tmp
drwxr-xr-x  14 root root      4096 May 20 22:05 usr
drwxr-xr-x  12 root root      4096 May 20 22:06 var
```

Within the web directory, we can see index.html and files.has, which are the two pages that the user has already seen. The files do_not_overwrite.has and upload.has are new, as well as the folder script_backups.

do_not_overwrite.has simply contains a string "WORK IN PROGRESS"

upload.has contains a simple upload form that lets the user upload a file to the /usr/local/usrfiles directory, and shows a "upload finished" message on uploading a file. (This change is mirrored on files.has too)

```
/usr/local/usrfiles
total 4
-rw------- 1 root root 327 Jun 18 14:07 test.py
```

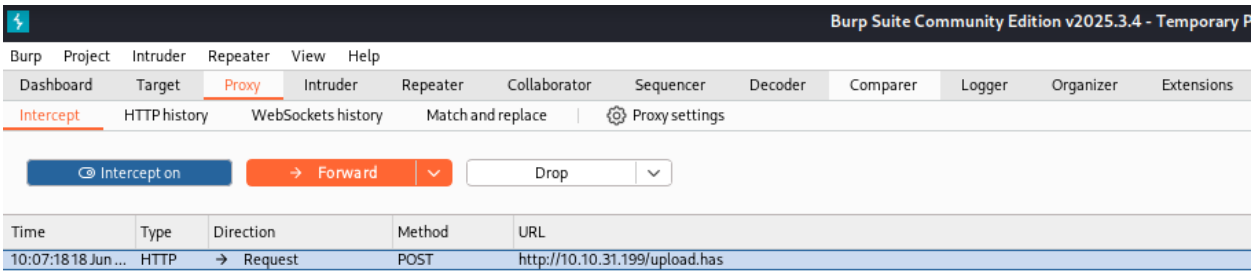
There doesn't seem to be any file type or content validation for files uploaded through this tool.

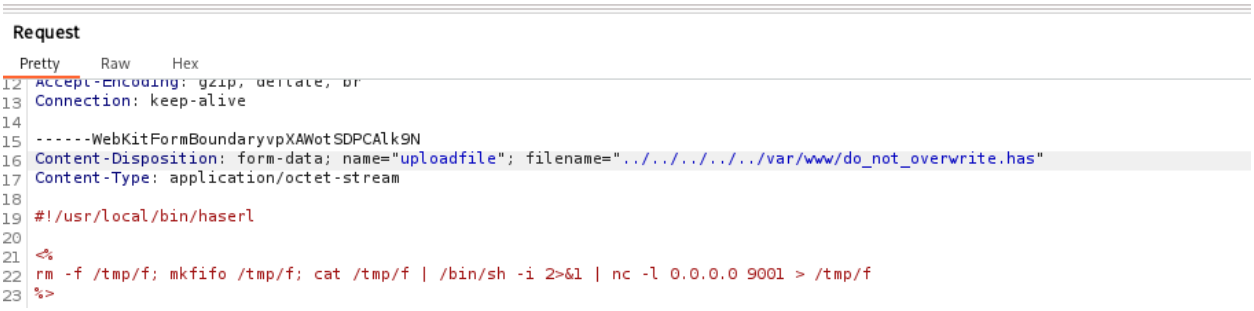
Running DIRB against the script_backups directory find files.txt, a file that seems to be the source code for files.has. The source code for all the other files can also be found within this directory, by simply replacing the .has extension with .txt, and checking out upload.txt reveals how the uploading feature is implemented, and that it is vulnerable to file path traversal.

```
┌──(kali㉿kali)-[~]
└─$ dirb http://10.10.31.199/script_backups /usr/share/wordlists/dirb/common.txt -x /usr/share/wordlists/dirb/extensions_common.txt -N 301

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Wed Jun 18 09:10:28 2025
URL_BASE: http://10.10.31.199/script_backups/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
OPTION: Ignoring NOT_FOUND code -> 301
EXTENSIONS_FILE: /usr/share/wordlists/dirb/extensions_common.txt | ()(.asp)(.aspx)(.bat)(.c)(.cfm)(.cgi)(.com)(.dll)(.exe)(.htm)(.html)(.inc)(.jhtml)(.jsa)(.jsp)(.log)
.shtml)(.sql)(.txt)(.xml)(/) [NUM = 29]

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://10.10.31.199/script_backups/ ----
+ http://10.10.31.199/script_backups/files.txt (CODE:200|SIZE:220)
```

```
10.10.31.199/script_backups/ ×   File upload!   ×   +
← → ↻ ⌂   10.10.31.199/script_backups/upload.txt

#!/usr/local/bin/haserl --upload-limit=4096 --upload-dir=/tmp

<%# --- Process the form submission --- %>
<%
if [ "$FORM_button" = "Upload_File"]; then
        if test -n "$HASERL_uploadfile_path"; then
                file_name=$FORM_uploadfile_name
                        if [ "$file_name" != "" ]; then
                                savedfile=$HASERL_uploadfile_path
                                cp $savedfile /usr/local/usrfiles/$file_name
                                up_status="<span class='impactOK'>Upload Finished </span>"
                        else
                                up_status="<span class='impact'>Invalid name</span>"
                        fi
        else
                up_status="<span class='impact'>   NULL file! </span>"
        fi
fi
%>
<%# --- Present the HTML page --- %>
<%
%>
<html lang="en">
<head>
<title>File upload!</title>
</head>
<body>
<form id="upload" action="<% echo -n $SCRIPT_NAME %>" method="POST" enctype="multipart/form-data">
<label>Upload File!</label>
<input type=file name=uploadfile>
<% echo $up_status %>
<input type="submit" name="button" value="Upload_File">
</form>
</body>
</html>
```

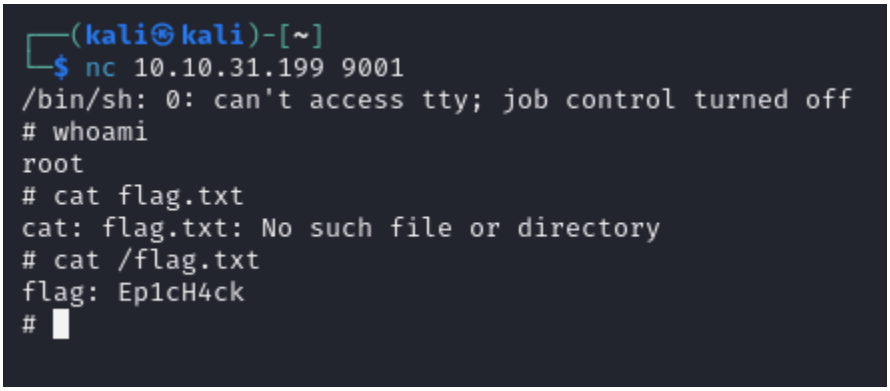
A bind shell script can be run using haserl, but regular reverse shell scripts will not work. Because the upload feature uploads to /usr/local/usrfiles with the permissions “rw-------”, a any haserl scripts uploaded will not be runnable, unless they overwrite an existing file. This is because when an existing file is overwritten, the permissions do not change.





Changing the upload request using burp proxy to include path traversal characters in filename.

Accessing do_not_overwrite.has and then opening a nc connection to the now open port.



Note: A bind shell isn't needed. A haserl script that simply runs "cat /flag.txt" would also give the same result.